Single-Axis Actuator: Requirements-Driven Control and Verification

A systems-engineering study of a motion-control subsystem, from needs to verified requirements

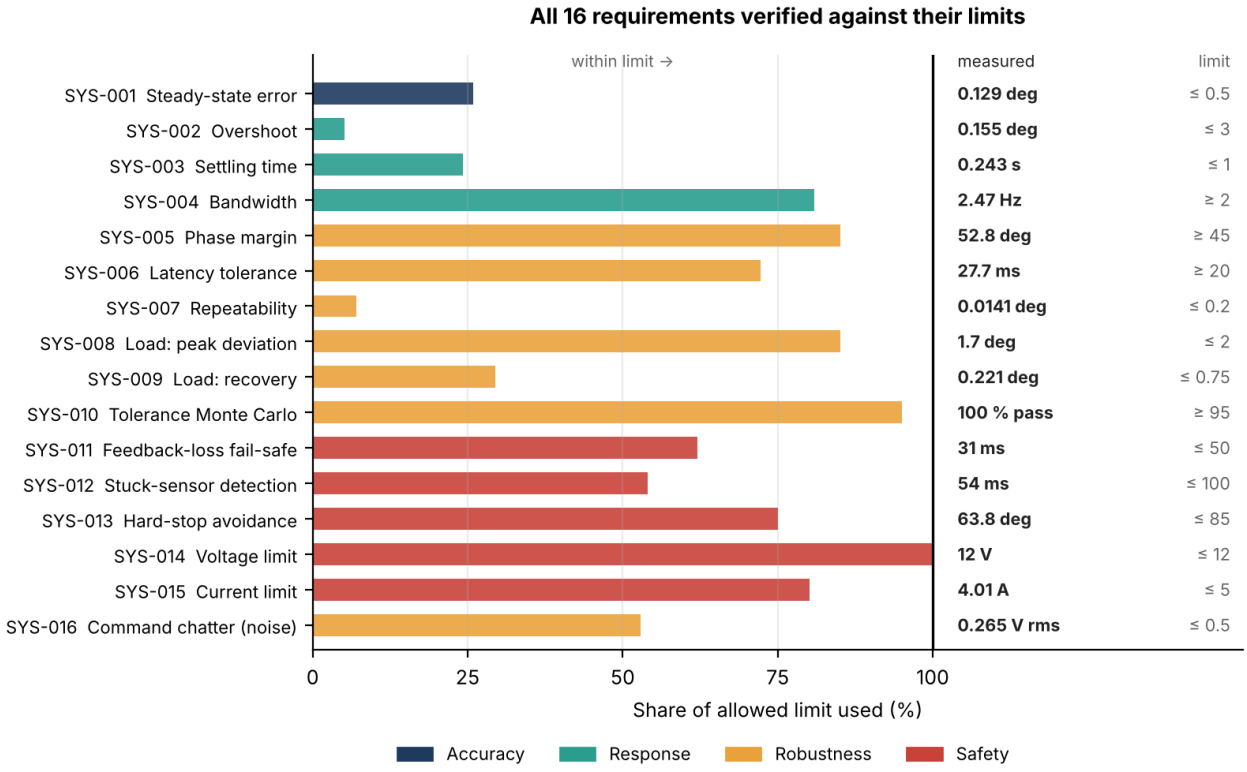
Daisy Wang | M.S. Mechanical Engineering (USC) | Python, linear analysis, Monte Carlo, fault injection | **Simulation-only project; requirement values are notional**

16 / 16
requirements verified, each traced to a need and a test

100%
of 200 randomised hardware/load variants meet SYS-001 to 004

8
issues found by testing and closed (Section 7)

7.0 A → 4.01 A
blocked-rotor current vs 5 A driver rating



What this project demonstrates

Requirements engineering	4 needs → 16 verifiable “shall” requirements with rationale, method and traceability (Section 2); one documented requirements trade (CR-1).
Verification and validation	Simulated test cases plus analysis (Bode margins, Monte Carlo), pass/fail with margins (Sections 3–5).
Failure thinking	Fault injection exposed hazards the first draft did not handle; defence-in-depth fix verified (Section 5).
Trade studies and iteration	Design-space map, three design iterations, honest record of what failed (Section 6).
Troubleshooting and documentation	Issues log with evidence, root cause, fix; known limitations and a hardware-transition plan (Sections 7–8).

1 System context and architecture

Operational need (notional). A small rotary actuator positions a load (a control surface or jet-vane-class element) from commands issued by an outer guidance loop running near 0.5 Hz. The actuator must be accurate, fast enough for that loop, robust to part variation, load and latency, and must fail safe. The four needs drive every requirement: **N1 Accuracy**, **N2 Response**, **N3 Robustness**, **N4 Safety**.

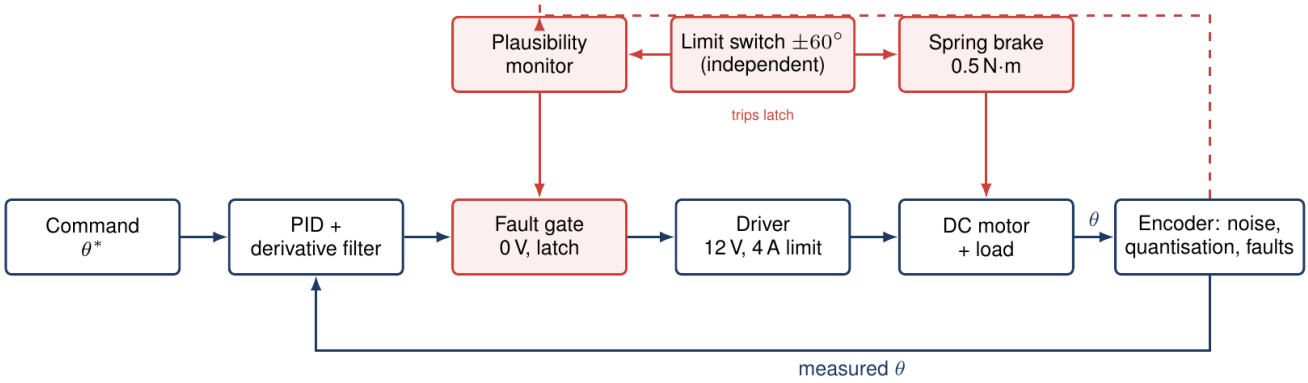


Figure 1. Architecture. Blue: control path. Red: safety layers, with the limit switch and brake independent of the encoder and controller.

Plant model. Armature circuit and rigid-body dynamics, with Coulomb friction, a smooth brake torque and ±90° hard stops:

$$L \cdot di/dt = v - R \cdot i - K_e \cdot \omega$$

$$J \cdot d\omega/dt = K_t \cdot i - b \cdot \omega - \tau_c \cdot \tanh(\omega/\omega_0) - \tau_load - \tau_brake \cdot \tanh(\omega/\omega_b)$$

$$d\theta/dt = \omega$$

Neglecting Coulomb friction and saturation gives the linear plant used for margin analysis, and the loop with a filtered derivative on measurement:

$$P(s) = K_t \,/\, (s \cdot [\, (J \cdot s + b)(L \cdot s + R) + K_t \cdot K_e \,])$$

$$L(s) = (K_p + K_i/s + K_d \cdot s/(\tau_d \cdot s + 1)) \cdot P(s)$$

$$T(s) = (K_p + K_i/s) \cdot P(s) \,/\, (1 + L(s))$$

Phase margin is PM = 180° + ∠L(jω_c) at gain crossover |L(jω_c)| = 1. The tolerable pure delay before instability is the delay margin:

$$t_d,max = PM\ [rad] \,/\, \omega_c$$

2 Requirements and traceability

Every requirement has a parent need, a rationale (for example SYS-004 flows down from a 4× separation to the 0.5 Hz outer loop), a verification method and a pass/fail limit in one registry (requirements.py) that also drives the tests and this table.

Need	ID	Requirement	Method	Limit	Measured	Margin	Result
N1	SYS-001	Steady-state error	Test	≤ 0.5 deg	0.129 deg	74%	PASS
N2	SYS-002	Overshoot	Test	≤ 3 deg	0.155 deg	95%	PASS
N2	SYS-003	Settling time	Test	≤ 1 s	0.243 s	76%	PASS
N2	SYS-004	Bandwidth	Analysis	≥ 2 Hz	2.47 Hz	24%	PASS
N3	SYS-005	Phase margin	Analysis	≥ 45 deg	52.8 deg	17%	PASS
N3	SYS-006	Latency tolerance	Analysis + Test	≥ 20 ms	27.7 ms	38%	PASS
N3	SYS-007	Repeatability	Test	≤ 0.2 deg	0.0141 deg	93%	PASS
N3	SYS-008	Load: peak deviation	Test	≤ 2 deg	1.7 deg	15%	PASS
N3	SYS-009	Load: recovery	Test	≤ 0.75 deg	0.221 deg	71%	PASS
N3	SYS-010	Tolerance Monte Carlo	Analysis	≥ 95 % pass	100 % pass	5%	PASS

Need	ID	Requirement	Method	Limit	Measured	Margin	Result
N4	SYS-011	Feedback-loss fail-safe	Test	≤ 50 ms	31 ms	38%	PASS
N4	SYS-012	Stuck-sensor detection	Test	≤ 100 ms	54 ms	46%	PASS
N4	SYS-013	Hard-stop avoidance	Test	≤ 85 deg	63.8 deg	25%	PASS
N4	SYS-014	Voltage limit	Test	≤ 12 V	12 V	by design	PASS
N4	SYS-015	Current limit	Test	≤ 5 A	4.01 A	20%	PASS
N3	SYS-016	Command chatter (noise)	Test	≤ 0.5 V rms	0.265 V rms	47%	PASS

Margin = distance from the limit as a share of the limit. SYS-014 is met by voltage saturation, so zero margin is by construction. SYS-010 can only exceed its limit by 5 points. Full text and rationale: results/traceability_matrix.csv.

3 Verification approach and nominal results

Each case is a repeatable, seeded simulation with a defined input, measurement and criterion (verification.py). Nominal 30° step: overshoot 0.16°, settling in 0.24 s against a 1.0 s limit, peak current 3.5 A (under the 4 A limit), steady-state command activity 0.26 V rms. A 0.025 N·m load applied for one second moves the axis 1.70° at worst, and it recovers to within 0.22°.

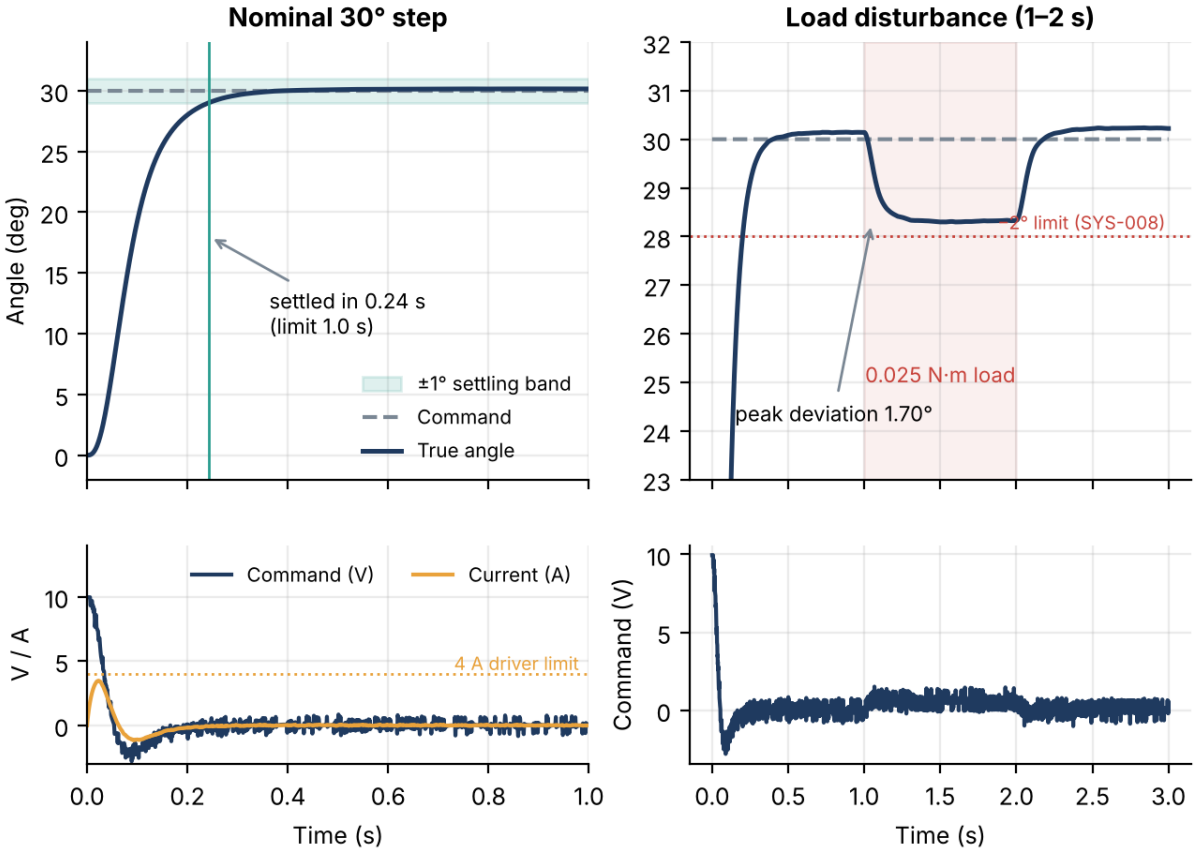


Figure 2. Left: nominal step with settling band and electrical limits. Right: load-disturbance test against the SYS-008 limit.

4 Frequency-domain analysis and latency

Loop crossover is 5.3 Hz with 53° phase margin and a 28 ms delay margin. Closed-loop bandwidth is 2.47 Hz against the 2.0 Hz requirement.

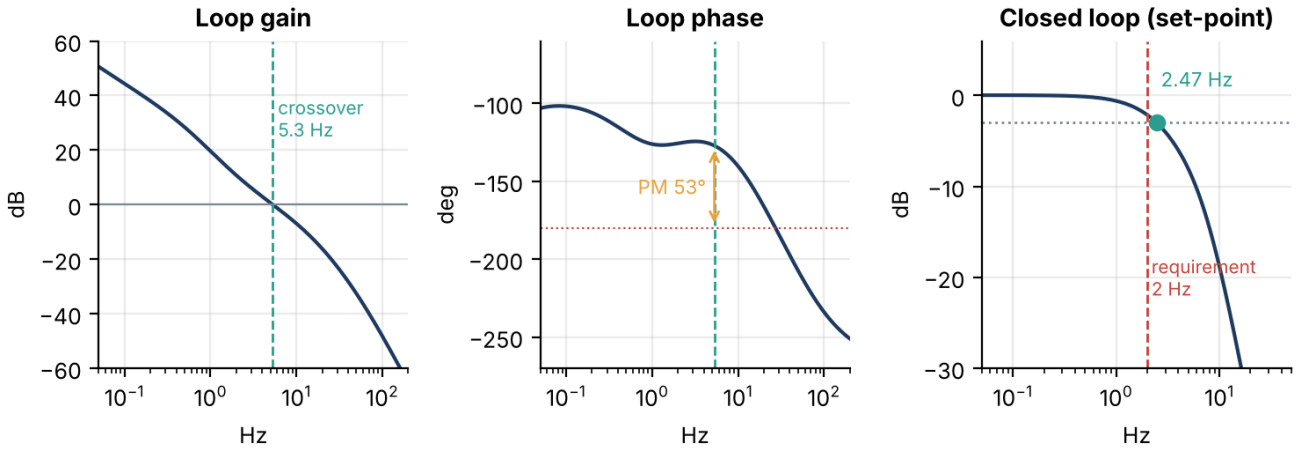


Figure 3. Bode analysis of the final design (Kp 19, Ki 1, Kd 1.6, 3 ms derivative filter).

The latency sweep checks the analysis against simulation: overshoot grows sharply as added delay approaches the predicted delay margin.

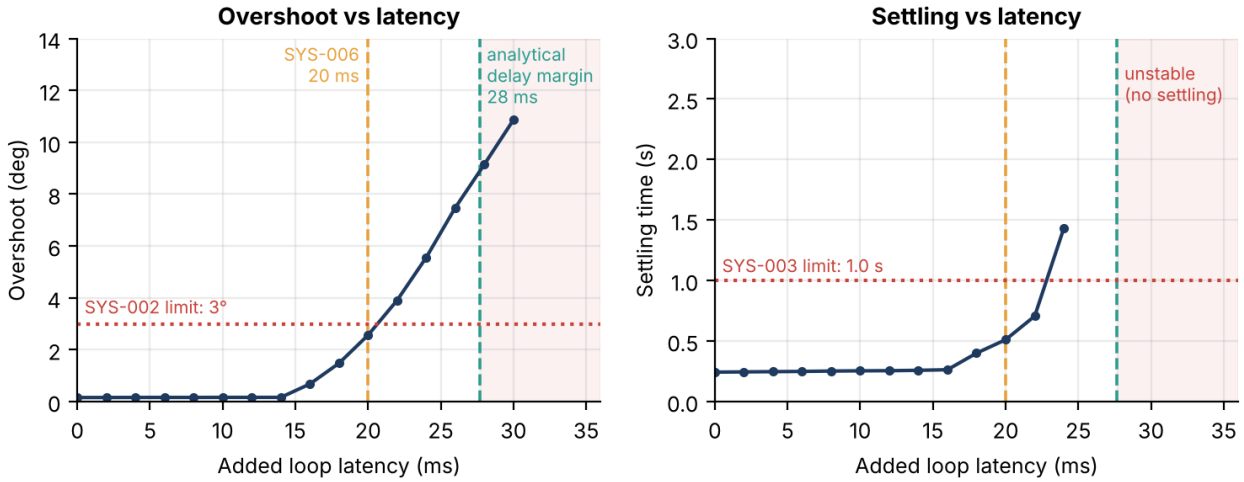


Figure 4. Added loop latency vs overshoot and settling, compared with the analytical delay margin.

5 Fault management and defence in depth

Fault injection (frozen encoder, total and intermittent dropout, invalid readings, blocked rotor) showed that no single protective measure was enough. The final design uses independent layers:

Layer	Detects	Response
Bad-sample filter	dropout / invalid reading	0 V on that sample (SYS-011)
Loss timeout (20 ms)	feedback gone	latch fault, brake at 30 ms (SYS-011)
Plausibility monitor	command > 1 V with no encoder motion for 50 ms	latch fault, brake (SYS-012)
Limit switch ±60°	travel overrun, independent of encoder	latch fault, brake (SYS-013)
Driver current limit (4 A)	overcurrent, blocked rotor	clamp voltage (SYS-015)

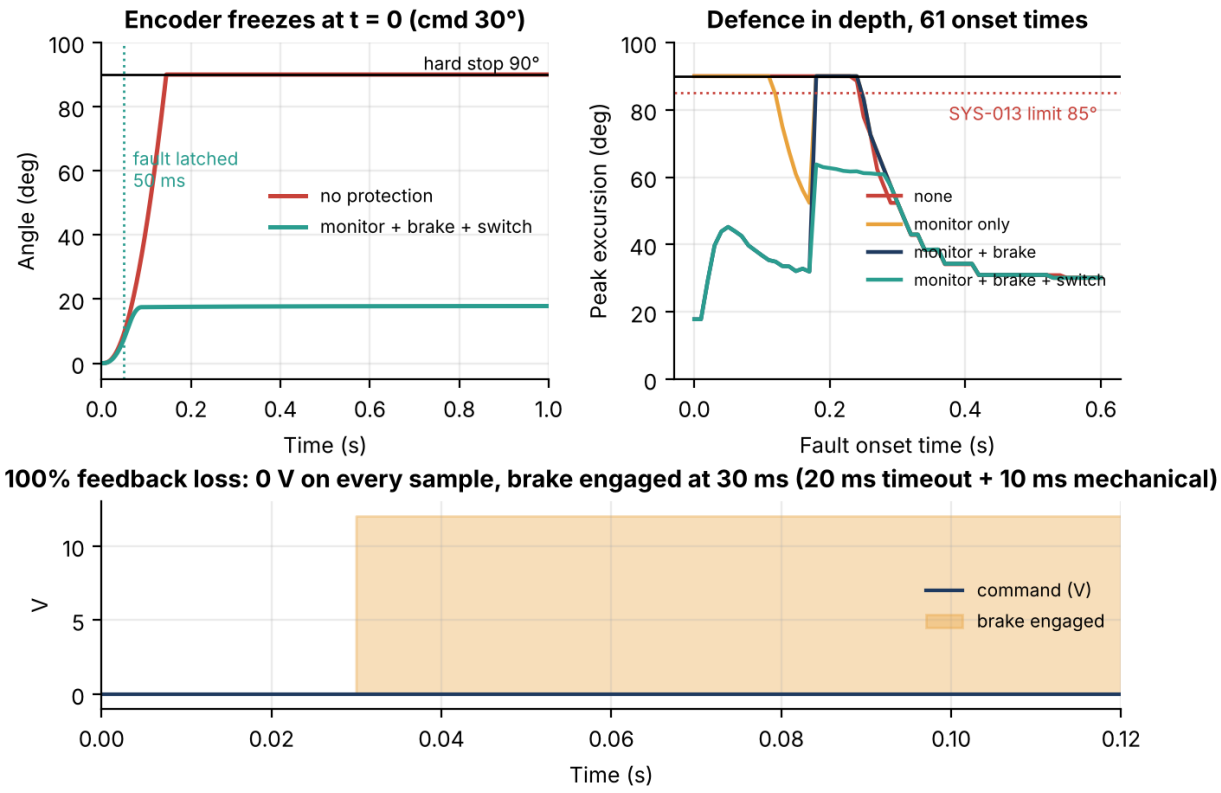


Figure 5. Top left: frozen encoder at t = 0. Top right: worst-case excursion over 61 fault-onset times as layers are added. Bottom: total feedback loss.

The monitor and brake alone still reached the hard stop for fault onsets of about 0.18 to 0.24 s, just as the axis arrives at target and the frozen reading looks nearly correct. The independent limit switch closes that gap (needed in 11 of 61 onsets), giving a worst case of 64° against the 85° limit.

6 Design iterations and requirements trade

Three designs were verified against the same 16 requirements. Gains are Kp / Ki / Kd / derivative-filter constant.

Design	Kp / Ki / Kd / filter	BW (Hz)	PM	Delay margin	Verified	Failed requirements
Iter 0 (as submitted)	14 / 1 / 1.2 / none	2.50	59°	40 ms	14/16	SYS-008, SYS-016
Iter 1	24 / 1 / 1.6 / none	3.51	53°	28 ms	14/16	SYS-006, SYS-016
Final	**19 / 1 / 1.6 / 3 ms**	**2.47**	**53°**	**28 ms**	**16/16**	**—**

Iteration 0 (the submitted gains) misses SYS-008 (2.30° vs 2.0°) and SYS-016 (1.3 V rms chatter). Iteration 1 raised the gains for load stiffness and fails SYS-006 (5.6° overshoot at 20 ms latency, even though the delay margin looked healthy). The final design is the interior of the feasible region found by a grid search.

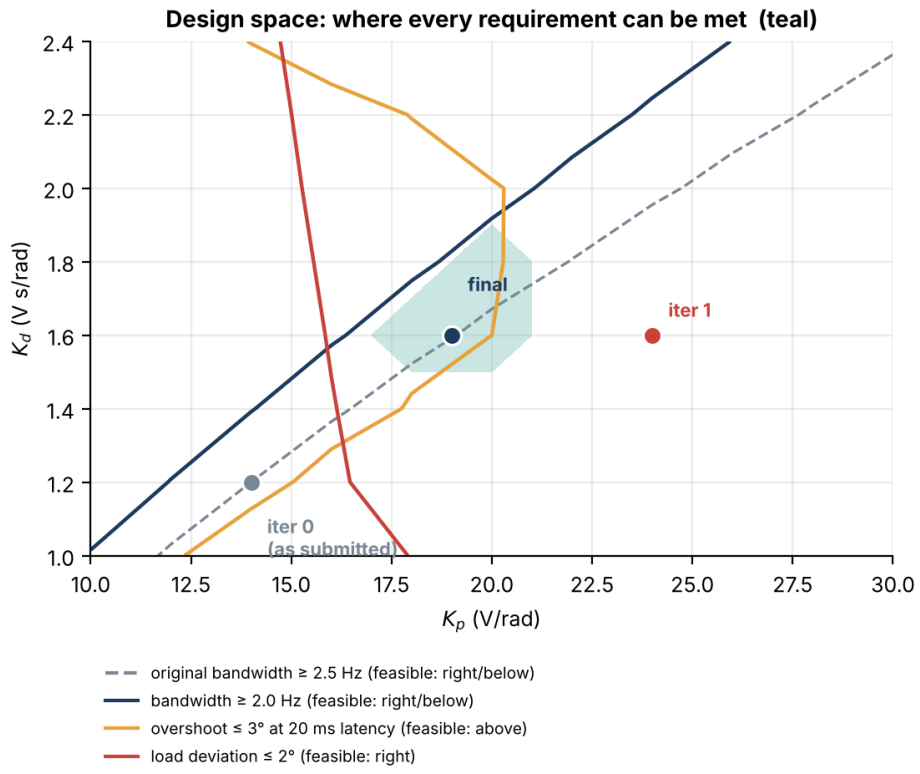


Figure 6. Feasible region in (Kp, Kd) for all nominal requirements.

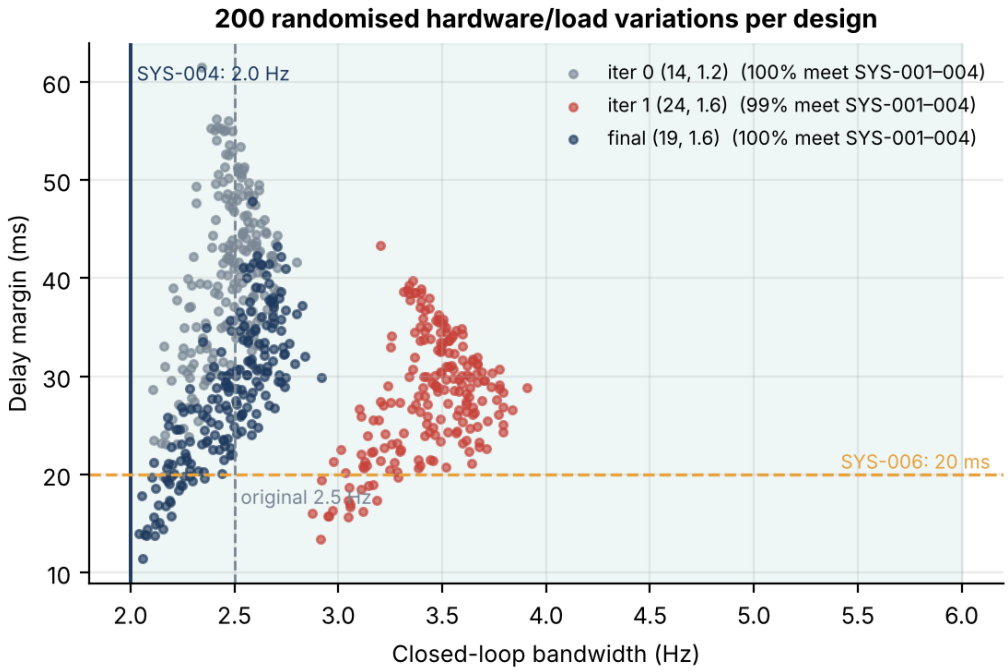


Figure 7. Bandwidth vs delay margin over 200 randomised variants per design.

Change request CR-1 (bandwidth 2.5 Hz → 2.0 Hz). The original 2.5 Hz limit (5× the outer loop) cut through the feasible region (grey dashed line in Figure 6), leaving only its lower-right corner, which sits next to the latency-overshoot boundary with thin margin. I relaxed it to 2.0 Hz (4×, still inside the 4 to 10× rule of thumb), which puts the final design well inside the region with at least 15% margin on every nominal requirement. The final bandwidth of 2.47 Hz would still miss the original 2.5 Hz by about 1%. This is a requirement trade made with evidence and recorded, not a silent edit.

7 Issues log

ID	Finding (evidence)	Root cause	Fix and re-verification
1	Frozen encoder drives axis to the 90° hard stop (90° peak, no protection)	PID trusts a sensor that still “looks healthy”	Plausibility monitor; SYS-012 verified (54 ms)
2	Removing drive alone did not stop the axis	Low back-EMF damping vs inertia: it coasts	Spring-applied fail-safe brake; excursion 18° at t = 0 onset
3	Monitor + brake still hit the hard stop for onsets near arrival at target	Frozen reading near setpoint gives a small command	Independent ±60° limit switch; worst case 64°
4	Blocked-rotor current 7.0 A vs 5 A rating	Full 12 V plus back-EMF during hard reversal	4 A driver limit (80% derating); 4.01 A measured
5	Command chatter of 1.3 V rms at steady state	Derivative term amplifies 0.088° encoder quantisation at 1 kHz	3 ms derivative filter; 0.26 V rms; new requirement SYS-016
6	28 ms delay margin but 5.6° overshoot at 20 ms latency	Stability margin is not performance margin	Added the time-domain latency test to SYS-006; retuned
7	Bandwidth (2.5 Hz) and latency tolerance leave a thin-margin corner	Requirements conflict, found via design-space and Monte Carlo	CR-1 (Section 6)
8	Draft suite passed with huge margins and had code defects	Nominal-only tests; literal \n printed; test named “six” asserted eight	Registry-driven suite, tolerance analysis, tests fixed

8 Limitations and next steps

- **Simulation only.** No hardware was built; conclusions are about the model. Requirement limits are notional and should be replaced with program values.
- **Latency tolerance across tolerances.** SYS-006 is verified at nominal parameters; over the 200-run tolerance set the delay margin was at least 20 ms in 85.5% of cases. A reviewer should treat this as the weakest requirement.
- **Nuisance trips.** A sustained static load of about 0.05 N·m (first trip in a 0.005 N·m scan) is read as stall and trips the monitor. Distinguishing stall from a frozen sensor needs a second signal, such as a back-EMF speed estimate (limited by resistance tolerance) or an encoder health flag.
- **Not modelled:** thermal limits, gear backlash and compliance, driver PWM and noise coupling, real encoder protocols, aerodynamic hinge moments.

Hardware-transition plan. (1) Identify motor and inertia from a bench step and friction test and re-run the Monte Carlo with measured spreads. (2) Log command, encoder and current at a fixed rate through the same CSV format used here and compare against model predictions. (3) Repeat the fault-injection suite on a protected bench setup, starting with the dropout and frozen-sensor cases. (4) Write the test procedures as controlled documents with pass/fail criteria taken from the requirement registry.

Reproduce. python run_project.py (verification suite and results CSVs), python report/make_report_assets.py (all figures and data), pytest (unit and requirement tests). All figures and tables in this report are generated from the code.